

# Class members (Indexer)

عشان نفهم ال **indexer** لازم الاول نفهم يعني اي **index**

لو عندي

```
int[] arr = { 1, 2, 3 };
var num = arr[0];

string s = "Menna";
var c = s[0];
```

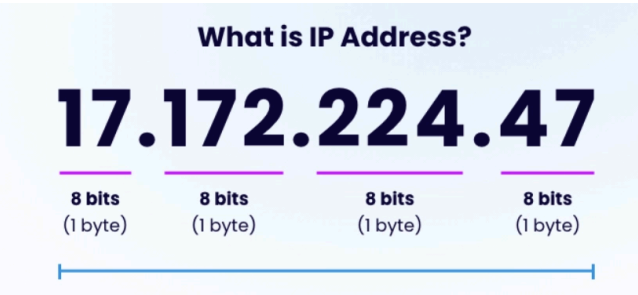
ال **Array** او **List** او **String** عبارة عن مجموعات (مكونه من اكثر من عنصر عشان كد اقدر اطبق عليها ال **index** )

طب لو انا عايز اعمل حاجه زي كد على ال **Object** هنا ال **#C** فيها حاجه بتخليني اعمل كد وهيا ال **Indexer**

## ما هو ال-Indexer؟

ال- **Indexer** هو ميزة في **#C** تسمح لك بالتعامل مع كائن (**Object**) وكأنه **Array** أو **List**.

مثال لو عندي برنامج ال **IP Address** ال **IP Address** عبارة عن اربع اجزاء



دلوقتي لو انا عايز اعمل **Modeling** لل **IP Address**

وعايز بدل م اعمل **Method** ترجعلي **one segment** لا انا عايز اطبق ال **indexer** و اقدر اعمل حاجه زي كد

```
var ip = new IP(112, 345, 554, 45);
```

```
var firstsegmentip1 = ip[0];
```

## شكل ال-Indexer

ال- **Indexer** هو نوع خاص من ال- **Property** يسمح باستقبال **Parameters** واستخدام الأقواس **[]** للوصول إلى البيانات.

```
public dataType this[indexType index]
{
    get //get(index)
    {
        // Return value
    }

    set //set(index,value)
    {
        // Assign value
    }
}
```

**dataType**

بتشير الى ال **field** اللي بعمل ليها ال **indexer** يعني في ال **IP** هيكون **segments array** يعني **int**

**indexType**

في معظم الوقت هيكون **int** لاني بتعامل مع **index**, مش دائما **int**

بيق **Modeling** ال **IP Address** هيكون كد

```
s    static void Main(string[] args)
{
    var ip = new IP(112, 345, 554, 45);
    var ip2 = new IP("111.345.677.89");
    var firstsegmentip1 = ip[0];
    Console.WriteLine(ip.Address);
    Console.WriteLine(ip2.Address);
    Console.WriteLine(firstsegmentip1);
    var firstsegmentip2 = ip2[0];
```

```
        Console.WriteLine(firstsegmentip2);
        Console.ReadKey();
    }

}

public class IP
{
    public int[] Segments = new int[4];
    //Indexer
    public int this[int index]
    {
        get
        {
            return Segments[index];
        }
        set
        {
            Segments[index] = value;
        }
    }
    // string او ك Segments بطريقتين ك مجموعة من ال id باخذ ال
    // As four Segments
    public IP(int segment1, int segment2, int segment3, int segment4)
    {
        Segments[0] = segment1;
        Segments[1] = segment2;
        Segments[2] = segment3;
        Segments[3] = segment4;
    }
    // As one string
    public IP(string ipaddress)
    {
        var segs = ipaddress.Split('.');
        for (int i = 0; i < segs.Length; i++)
        {
            Segments[i] = Convert.ToInt32(segs[i]);
        }
    }
    public string Address => string.Join(".", Segments); // ال id return بت
}
```

ماذا يحدث خلف الكواليس؟

لما بنكتب:

```
ip[0];
```

ال compiler يحولها الى:

```
ip.GetItem(0);
```

ولو عايزين

```
ip[0] = 55;
```

فإنها تتحول إلى:

```
ip.setItem(0, 55);
```

## Indexer Read Only

لو عايزينه Read Only

```
public int this[int index]
{
    get
```

```
        {
            return Segments[index];
        }

    }
}
```

لا يمكن التعديل:

```
ip[0] = 55; // Error
```

## String باستخدام Indexer

مثال:

```
class Employee
{
    private Dictionary<string, string> data =
        new Dictionary<string, string>();

    public string this[string key]
    {
        get
        {
            return data[key];
        }

        set
        {
            data[key] = value;
        }
    }
}
```

الاستخدام:

```
Employee emp = new Employee();

emp["Name"] = "Menna";
emp["City"] = "Mansoura";

Console.WriteLine(emp["Name"]);
```

النتائج:

```
Menna
```

## Multiple Parameters

يمكن أن يحتوي الـ Indexer على أكثر من Parameter.

مثال:

```
static void Main(string[] args)
{
    int[, ] sudokuMatrix = new int[9, 9]
    {
        { 5, 3, 0, 0, 7, 0, 0, 0, 0 },
        { 6, 0, 0, 1, 9, 5, 0, 0, 0 },
        { 0, 9, 8, 0, 0, 0, 0, 6, 0 },

        { 8, 0, 0, 0, 6, 0, 0, 0, 3 },
        { 4, 0, 0, 8, 0, 3, 0, 0, 1 },
        { 7, 0, 0, 0, 2, 0, 0, 0, 6 },

        { 0, 6, 0, 0, 0, 0, 2, 8, 0 },
    }
```

```
        { 0, 0, 0, 4, 1, 9, 0, 0, 5 },
        { 0, 0, 0, 0, 8, 0, 0, 7, 9 }
    };

    var suduko = new Suduko(sudokuMatrix);
    Console.WriteLine(suduko[1,4]); //9
        suduko[1,4] = 10;
        Console.WriteLine(suduko[1,4]); //10
    Console.ReadKey();
}

}

public class Suduko
{
    private int[,] _matrix;
    public int this[int r,int c ]
    {
        get
        {
            return _matrix[r,c];
        }
        set
        {
            _matrix[r, c] = value;
        }
    }
    public Suduko(int[,] matrix)
    {
        _matrix = matrix;
    }
}

}
```

كمان اقدر اضيف **Validation** عادي

## Indexer Overloading

اقدر اعرف اكثر من indexer و ال compiler هيقدر يحدد انا عايز اي واحد.

اذن انا بستخدم ال **indexer** دائما لما ال **object** بيكون فيها مجموعات عشان اسهل التعامل مع القيم.